E - Riverside Curio

Process

The question said a person is marking each day's water level. He only marks the water level if there is no old water level in the same position. And he will record the number of marks strictly above the current day's water level.

The question aimed to find the minimum possible sum of the number of marks strictly below the water level among all days.

Challenges and Overcoming

In the start, I have no idea of how to do this question, especially how to get the number of marks strictly below the water level in each day.

After getting some hits, because what we know is the number of marks above the water level, in order to get the number of marks strictly below the water level, we need to find something that links number of marks above and below together. A nature way of thing that is linking them by the number of total marks for each day.

Clearly, without the constrain of the person only marks the water level if there is no old water level in the same position, the number of total marks is just the number of days. However, since we have the constrain, the rule we have will be: Total[i - 1] <= Total[i] <= Total[i - 1] + 1. Besides, since we know there are Above[i] marks plus current water level for sure, there is Total[i] >= Above[i] + 1.

And clearly, Total[i] = Above[i] + Below[i] + 1 => Below[i] = Total[i] - Above[i] - 1.

Another challenge is you can not make every constrian for Total satisfy in one loop.

```
for (int i = 1; i < n; i++) {
    int prev = total[i - 1];
    int curr_and_above = m[i] + 1;
    total[i] = max(prev, curr_and_above);
}

for (int i = n - 2; i >= 0; i--) {
    total[i] = max(total[i + 1] - 1, total[i]);
}
```

We have to check if the previous day's total mark can have a result as today's mark, since we can only add 1 mark more in one day.

The rest is easy, just use Below[i] = Total[i] - Above[i] - 1 to calculate the sum of the number of marks strictly below the water.